Collate — Architecture (v1)

Version 0.1 · 31 Oct 2026 · Owner: Julian Duru · Author: Julian Duru · Status: Draft for review

This document tells how we build Collate v1. The PRD ("Collate — Product Requirements (v1)") gives the requirements and the terms. Each choice has a tag: **Decided** or **Recommended**.

1. Scope and principles

The current code is a frontend prototype (`Collate-Africa/collate`, commit `cadde09`) with mock data only. Stack: Next.js 16.4.3, React 19.3, Tailwind 4, shadcn on Base UI, npm.

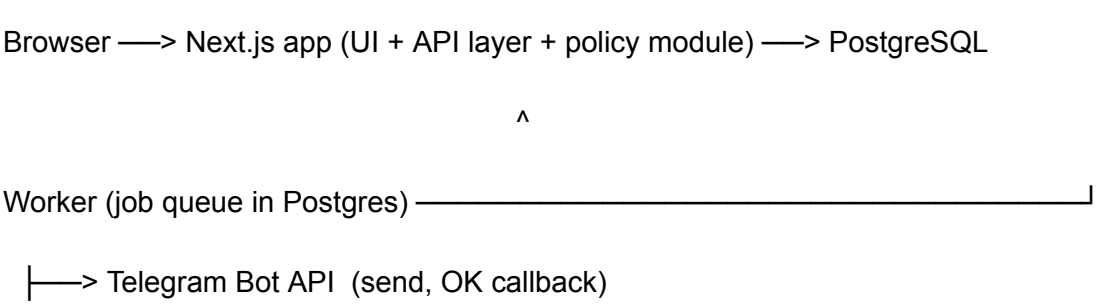
Principles:

1. Simple first. Use few components and one database.
2. Scope-safe by default. The server applies the scope to each query. The UI never decides access.
3. Extensible by data, not code. Offices, election days and visibility rules are rows in tables.

2. System overview

Component	Function	Tag
Web app	The current Next.js app, in the same TypeScript repository as the backend	Recommended
API layer	Server code in the Next.js app. It contains the policy module.	Recommended
Worker	A separate process for broadcasts, imports and coverage refresh. It uses a Postgres job queue (for example pg-boss or Graphile Worker).	Recommended
PostgreSQL	The one database. PostGIS is optional, for map data later.	Recommended
Telegram	The primary channel	Decided

Component	Function	Tag
SMS	The fallback channel. Not decided: provider.	Decided
Sentry	Error reports. Not set up yet.	Decided



3. Data model

All tables are in PostgreSQL. Names can change during the build.

Table	Key columns and relations
`state`, `lga`, `ward`	INEC code, name, parent code
`polling_unit`	INEC code (for example `04-17-03-012`), `ward` code, latitude, longitude, `position_doubtful`
`political_unit`	type (country, state, senatorial district, federal constituency, state constituency), parent
`pu_political_unit`	PU, political unit. Many-to-many.
`office`	name, level (rank in the tree), political-unit type
`election_day`	date, offices, off-cycle states
`seat`	office, political unit

Table	Key columns and relations
`candidate`	person, seat, `running_mate_of` (candidate)
`person`	name, home PU, email, PVC number, verification status
`phone_number`	person, E.164 number, `confirmed`, Telegram chat ID
`consent`	person, text version, time
`group`	name, type (party, campaign, support group), parent group
`membership`	person, group, role, scope units. Admins are members of one `admin` group. The role holds the admin level.
`group_visibility_grant`	viewer group, owner group, allowed
`agent_assignment`	person, PU, owner (group or candidate), kind (`official` or `extra`), slot
`broadcast`	sender, audience filter, text, schedule, status
`delivery`	broadcast, person, channel, status, times
`access_link`	candidate, token hash, expiry, revoked
`invitation`	inviter, phone number, name, group, role, scope units, token hash, expiry, status
`audit_log`	actor, action, target, time, IP address
`import_batch`	file, uploader, counts, errors
`coverage_count`	unit (administrative or political), owner, kind, PUs covered, PUs total

Constraints (Recommended): in v1, each PU has at most one `official` row. Each (PU, owner, slot) is unique. Section 10 gives the change for per-candidate official agents.

4. Authorization model

Scope resolution (Recommended). For each request, the policy module calculates:

1. The user's seats and group roles from `candidate` and `membership`.
2. The scope: the set of PU IDs from those seats and scope units, through `pu_political_unit` and the administrative tree. An admin gets all PUs.
3. The persons below the user: all agents at PUs in the scope, and users in the scope with a lower office or group level (PRD tree rule).
4. The visible owner groups from `group_visibility_grant`. v1 seeds "allow all" rows.

Enforcement point (Recommended). Each read and each broadcast goes through one server-side policy module. Each query must apply the scope filter and the grant filter. Route handlers do not query tables directly. Postgres row-level security is a later option.

Worked example. The Lagos governorship candidate logs in. The seat maps to all Lagos PUs. The grants allow all owners. Thus the candidate sees all agents in Lagos and gets no rows from other states. If an admin sets the grant "NDC national → Obi campaign" to not allowed, NDC national users no longer see Obi campaign agents. The code does not change.

5. Coverage computation

1. Headline coverage (Decided): PUs with an `official` row in `agent_assignment`.
2. Group coverage: PUs with at least one row for that owner. The count for a group includes its child groups (Recommended). Combined coverage: PUs with any row of the groups the user selects. Calculate child-group and combined coverage as a union of PUs, not as a sum of counts.
3. Precompute (Recommended): store the counts per unit and per owner in `coverage_count` (or materialized views). Do not count 176,846 PUs on each page load.
4. Refresh: the worker refreshes the counts after each import and each agent change.
5. Filter the counts by the user's scope and grants before display.

6. Messaging pipeline

1. **Broadcast.** The user selects the audience, writes the message and sets the schedule. The API stores a `broadcast` row.
2. **Recipient resolution.** The worker applies the audience filter inside the sender's scope. It adds the running mates of candidate recipients, removes duplicate persons and creates one `delivery` row per person.
3. **Queue.** The worker puts one job per delivery into the Postgres queue.

4. **Adapter.** Each channel adapter has `send()`. `receive()` is reserved for the inbound plug-in. Adapters: Telegram and SMS.
5. **Delivery status.** The worker sets each status: sent, delivered, delivered by SMS, failed, tapped OK. Telegram does not report receipt on the phone. Thus "delivered" means that the Telegram API accepted the message.

Rules (Recommended):

- **Rate limit.** Telegram permits about 30 messages per second for each bot. A national broadcast to more than 400,000 agents takes hours. The queue must throttle.
- **Retries.** The worker retries a failed send with a longer delay each time, up to a limit.
- **SMS fallback.** If a person has no Telegram link, or Telegram fails after the retries, the worker sends by SMS.
- **OK callback.** Each Telegram message has an inline OK button. The webhook sets the delivery to "tapped OK".

7. Identity and access

- **User invitation** (Recommended): no public sign-up. The policy module accepts an invitation only if its scope units are in the inviter's scope. Its role level must be lower than the inviter's level. Store only the token hash. When the invitee confirms the OTP, the API creates the `person`, `phone_number` (`confirmed`), `membership` and `consent` rows, and writes the event to `audit_log`.
- **User login** (Recommended): one-time password (OTP) by phone or email.
- **Candidate links** (Recommended): random tokens. Store only the token hash. Each link has an expiry and a revoked flag. Write each use to `audit_log`. Each deployment selects if links need OTP.
- **Telegram match** (Decided): the person taps "share my number". The bot normalizes the number to E.164 and finds it in `phone_number`. If there is no match, the bot sends nothing. A new number gets `confirmed` only after the person confirms it through a listed number. Before the first message, the bot shows the consent text and records the answer in `consent` (Recommended).

8. Data import

- **INEC master data.** Load states, LGAs, wards, PUs and the PU-to-political-unit mapping from the dataset that Oo has. Mark a PU as "position doubtful" when its GPS position is outside its LGA outline (GRID3).
- **Agent lists.** Import CSV or Excel files as an `import_batch`. The worker validates each row and reports errors.
- **De-duplication** (Recommended). Normalize each phone number to E.164. Use it as the match key for persons. If two lists conflict, the list from the candidate wins (Decided).

9. Security and data protection controls

The PRD gives the NDPA and Telegram requirements. The technical controls are:

- The policy module applies role scope to all data access (Recommended).
- The audit log records each read of agent data and each broadcast (Recommended).
- The UI masks phone numbers. Candidates get no bulk export (Decided).
- The database encrypts stored personal data (Decided).

10. Extension points

Change	**How the design supports it**
Inbound messages	Use `receive()` on the channel adapter. Add a handler for each command.
Payments to agents	Add tables that refer to `person` and `agent_assignment`.
Per-candidate official agents	Add an owner and `election_day` to the official slot. Then count headline coverage per candidate.
New deployments	Create a new database. Load offices, election days and groups as data.
Group restrictions	Change rows in `group_visibility_grant`.

11. Delivery plan

Each step is a vertical slice with tests, code review, merge and deploy (Decided 31 Oct). The order is Recommended.

1. Set up PostgreSQL, migrations, the worker, Sentry and one test pipeline.
2. Load the INEC master data. Replace the mock data in `lib/data/*.ts` for the coverage tree.
3. Add user invitations, user login and the policy module. Replace the demo personas in `lib/tree-permissions.ts`.
4. Add persons, agent assignments and the agent import.
5. Add coverage counts. Connect the Coverage, Candidates and Agents pages.
6. Add the Telegram bot: number match, consent, `send()` and the OK callback.

7. Add broadcasts: wizard, recipient resolution, queue and delivery status.
8. Add candidate links and the access log.
9. Add the home dashboard, admin settings and the audit log view.
10. Add the SMS adapter after the provider is decided.

12. Open items

Technical items only. The PRD lists the product items.

1. Not decided: hosting provider and region (NDPA data residency).
2. Not decided: SMS provider and its API.
3. To confirm: format of the INEC PU dataset, and if it contains the PU-to-political-unit mapping.
4. To confirm: owner of the Telegram bot account and the domain.
5. To confirm: what code from Collate versions 1 and 2 we can use again.